

Course: Advanced Algorithm and C++

Week:03

Department: Computer Science

We live under the shared quote

“ Batch 3 - 3 Departments ”

1

Supported by:

CADT
IDT វិទ្យាស្ថានបច្ចេកវិទ្យាឌីជីថល
Institute of Digital Technology

Organized by:

NEKT
GEN-3
Batch 3, 3 Departments



What We Will Cover in 5 Weeks

Foundation and Memory Management

W1

Sorting Algorithms

W2

Linked List

W3

Stack and Queue

W4

Project

W5



TABLE OF CONTENTS

Introduction to Linked Lists	01
Singly Linked List	03
Doubly Linked List	07
Circular Linked List	08
Comparing Linked Lists	09



Learning Objectives

By the end of this session, you will be able to:

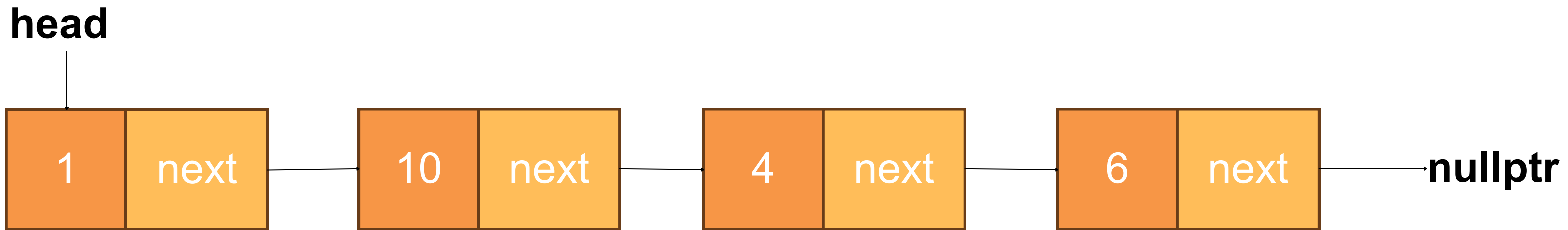
- **Explain** how a linked list stores data and how it differs from an array
- **Build** a Node class and a singly linked list with insert, search and delete operations
- **Traverse** a list safely and release every node when the list is destroyed
- **Compare** singly, doubly and circular lists and pick the right one for a problem

Introduction to Linked Lists

A linked list is a chain of nodes. Each node holds a value and a pointer to the next node, so the list can grow one node at a time

Key component of linked list are:

- **Node:** the value plus a pointer to the next node
- **Head:** a pointer to the first node in the list
- **Tail:** the last node, whose next is nullptr



! Never lose a linked list head pointer

Introduction to Linked Lists

There are three main types of linked list. They differ only in what each node points to

Singly linked list

Each node points forward to the next node. The last node points to nullptr

Doubly linked list

Each node points forward and backward, so the list can be walked in either direction

Circular linked list

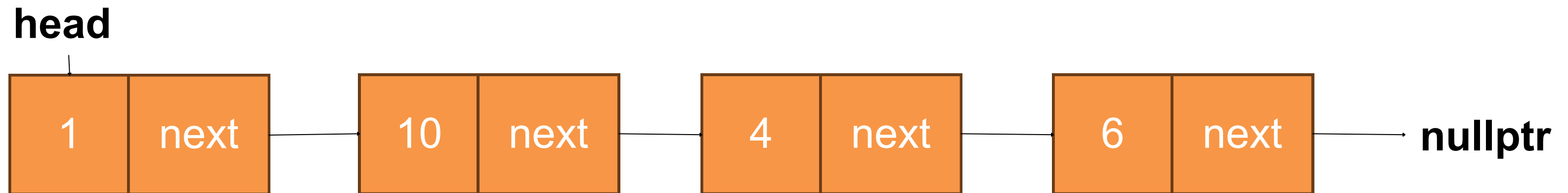
The last node points back to the head, so the chain has no end

Singly Linked List

A singly linked list (SLL) gives each node one pointer, to the node after it. Everything starts from the head and moves in one direction only

Cost of each operation

- Searching : $O(n)$ (must check each node in turn)
- Inserting at head: $O(1)$ (no walking, just repoint head)
- Inserting at end: $O(n)$ (must walk to the last node)
- Deleting a value: $O(n)$ (must find the node first)
- Traverse: $O(n)$ (must walk through all nodes)



Singly Linked List

The Node holds the data. The list class keeps the head private so nothing outside can reach into the chain

Step1: the Node

```
class Node {  
public:  
    int data;  
    Node* next;  
  
    Node(int value){  
        data = value;  
        next = nullptr;  
    }  
};
```

Step2: the List

```
class LinkedList {  
private:  
    Node* head;  
    int count;  
  
public:  
    LinkedList(){  
        head = nullptr;  
        count = 0;  
    }  
};
```


Singly Linked List

Traverse - $O(n)$

```
// walk from head to nullptr
void printList(){
    Node* cur = head;
    while(cur != nullptr){
        cout << cur->data << "->";
        cur = cur->next;
    }
    cout << "nullptr" << "\n";
}
```

Insert at the head - $O(1)$

```
void insertFront(int value){
    Node* newNode = new Node(value);
    newNode->next = head;
    head = newNode;
    count++;
}
```

Insert at the end - $O(n)$

```
void insertBack(int value){
    Node* newNode = new Node(value);
    if (head == nullptr){
        head = newNode;
        count++;
        return;
    }
    Node* cur = head;
    while(cur->next != nullptr){
        cur = cur->next;
    }
    cur->next = newNode;
    count++;
};
```

Order matters

Point newNode->next at head before moving head. Do it the other way round and the rest of the list is lost

Singly Linked List

Search - $O(n)$

```
bool search(int value){
    Node* cur = head;
    while(cur != nullptr){
        if(cur->data == value) return true;
        cur = cur->next;
    }
    return false;
}
```

Destructor

```
~LinkedList(){
    Node* cur = head;
    while(cur != nullptr){
        Node* nxt = cur->next;
        delete cur;
        cur = nxt;
    }
    head = nullptr;
}
```

Delete by value - $O(n)$

```
void remove(int value){
    if(head == nullptr) return;
    if(head->data == value){
        Node* dead = head;
        head = head->next;
        delete dead;
        count--;
        return;
    }
    Node* prev = head;
    while(prev->next != nullptr && prev->next->data != value){
        prev = prev->next;
    }
    if(prev->next == nullptr) return;
    Node* dead = prev->next;
    prev->next = dead->next;
    delete dead;
    count--;
};
```

Three ways deletion goes wrong

- Reading `cur->next` after deleting `cur`, which is a dangling pointer
- Forgetting that the head is a special case
- Never checking whether the value is in the list at all

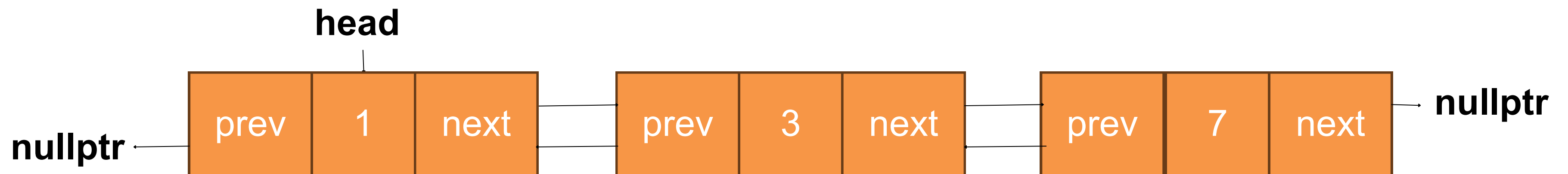
Doubly Linked List

A doubly linked list (DLL) gives each node a second pointer, prev so the chain can be walked in both directions

```
class Node {  
public:  
    int data;  
    Node* next;  
    Node* prev;  
  
    Node(int value){  
        data = value;  
        next = nullptr;  
        prev = nullptr;  
    }  
};
```

What the extra pointer can do

- **Walk backwards:** print the list in reverse without rebuilding it
- **Delete faster:** given a node, you already know the one before it
- **The cost:** one more pointer in every node, and every insert or delete updates twice as many links



Circular Linked List

A circular linked list (CLL) has no end. The last node points back to the head, so walking forward forever just goes round again.

This loops forever

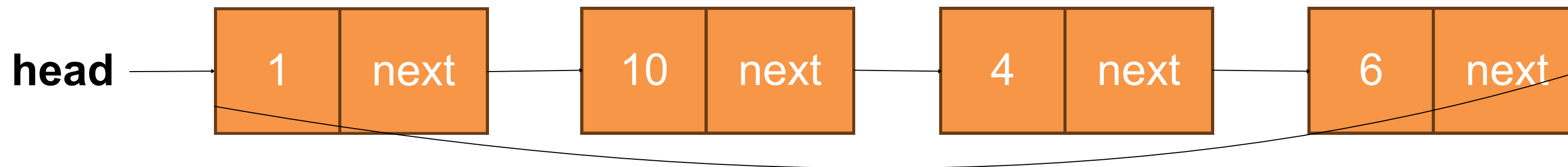
```
Node* cur = head;
while(cur != nullptr){
    cout<< cur->data;
    cur = cur->next;
}
```

This stops after one lap

```
if(head == nullptr) return;
Node* cur = head;
do {
    cout << cur->data << "->";
    cur = cur->next;
} while(cur != head);
```

When to go circular

- **Round-robin:** take turns forever without resetting to the start
- **Playlists:** the last track flows back to the first
- **Two variations:** circular singly (CSLL) and circular doubly (CDLL)
- **The catch:** no nullptr means no natural stop. Every loop needs a condition you write yourself



Comparing Linked Lists

Feature / Operation	Array	SLL	DLL	CLL
Size	Static	Dynamic	Dynamic	Dynamic
Reach item i	O(1) direct	O(n) walk	O(n) walk	O(n) walk
Insert/Delete at Front	O(n) shift all	O(1)	O(1)	O(1) with tail
Insert/Delete at End	O(1)	O(1) with tail / O(n) without tail	O(1) with tail	O(1) with tail
Walk backwards	Yes	No	Yes	Only if it's doubly
Memory Overhead	None	1 pointer/node	2 pointers/node	1 pointer/node

Lab Exercise

1. Count Nodes

Write **int countNodes()** that walk through the list once and returns how many node it has. Build the list of your choice, print the count and confirm it matches.

2. Find Min & Max

Write **void findMinMax()** that traverse through the list and return the smallest and largest value in the list and print both directly.

3. Sum All Values

Write **int sumAll()** that adds up every node's data into a total value in a single traversal and print the output.

4. Count Occurrences

Write **int countOccurences(int value)** that counts how many nodes match a given value and return that count.

Lab Exercise

1. Get Average

Write **double getAverage()** that sums the values and counts how many nodes in one traversal then return the average. Careful not to divide by zero if the list is empty.

2. Print Even & Odd Values

Write **void printEvenOdd()** that walks through the list and separates values by whether they're even or odd.

References

- [1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.
- [2] Deitel, P., & Deitel, H. (2017). C++ how to program (10th ed.). Pearson.
- [3] std::forward_list. (n.d.). cppreference.com.
https://en.cppreference.com/w/cpp/container/forward_list
- [4] std::list. (n.d.). cppreference.com. <https://en.cppreference.com/w/cpp/container/list>

The slide features decorative vertical bars on both the left and right sides. Each side has three bars of different colors: dark blue, green, and orange. The bars have a 3D effect with diagonal lines and a gradient. The text 'Thank You!' is centered in the middle of the slide in a large, orange, sans-serif font. A thin, light blue horizontal line is positioned directly below the text.

Thank You!